

SoK: Zero Trust as a strategy to address DevSecOps challenges

1st Anita Nair

Department of Computer Applications
Cochin University of Science and Technology,
Kochi, India
anita.nair@cusat.ac.in

2nd Serena Nicolazzo

Department of Computer Science
University of Milan
Milan, Italy
serena.nicolazzo@unimi.it

3rd Antonino Nocera

Department of Electrical, Computer,
and Biomedical Engineering
University of Pavia
Pavia, Italy
antonino.nocera@unipv.it

4th Rafidha Rehiman K. A.

Department of Computer Applications
Cochin University of Science and Technology,
Kochi, India
rafidharehimanka@cusat.ac.in

5th Vinod P.

Department of Computer Applications
Cochin University of Science and Technology,
Kochi, India
vinod.p@cusat.ac.in

Abstract—Over recent years, Development, Security, and Operations (DevSecOps, hereafter) has evolved as a unique shift-left strategy that integrates security practices, early on in the Development and Operations (DevOps, for short) software development methodology. However, DevSecOps is not devoid of challenges. Insecure tools and insider attacks are some of the threats that can escape the security controls enforced in DevSecOps pipelines. In this paper, we analyze the possibility of embedding Zero Trust into the organization's developer workflow as a strategy to further secure DevSecOps by removing the implicit trust among the various components of the pipelines. Based on a systematic review of the present literature and for each phase of the DevSecOps life cycle, we evaluated the possible integration of current ZTA approaches.

Index Terms—DevSecOps, CI/CD Pipeline security, Zero Trust Architecture, ZTA

1. Introduction

As IT organizations increasingly adopt DevOps methodologies for agile innovations, shifting security to the left is becoming an important consideration. The different stages of the DevOps cycle (Plan, Develop, Build, Test, Release and Deliver, Deploy, Operate, and Monitor) expose various types of vulnerabilities; which implies possible threats from the inside and the outside of the organization. The methodology of DevSecOps has emerged in recent years as an extension of DevOps, aiming at integrating security practices throughout the software development life cycle and addressing security concerns early in the development process. Although many companies have embraced this new security automation procedure in application development, several cybersecurity challenges still need adequate attention [Spahn et al.(2023)], [Rajapakse et al.(2022)]. One of the main drawbacks of the current DevSecOps systems is that once an attacker has access to the development pipeline via vulnerable components, such as untrusted third-party tools, they can laterally move

to other parts of the system to gain access to critical resources inside the network.

Zero Trust has gained importance in the scientific and industrial communities as a means of securing DevOps environments. It follows the principle of “Never Trust, Always Verify” and applies the best practices of least privilege, continuous verification using dynamic policies, micro-segmentation, and constant monitoring to decrease the attack surface effectively. In addition to securing the DevOps pipeline from external threats, the Zero Trust framework can also be adopted as a strategy for DevSecOps to reduce insider threats within organizations.

In this paper, although we do not aim to provide a complete systematic review of the literature, we mention relevant works that refer to the topics discussed in our analysis. To this objective, we consider journal and conference papers published during the last 10 years, in Google Scholar, Web of Science, ACM Digital Library, IEEE Xplore, USENIX, and SpringerLink. We also collected grey literature, which includes white papers and government reports. We included Q1 and Q2 journals, A-ranked conference papers, trending books, and high-quality white papers from government bodies, industry, and universities from Google Scholar. We excluded duplicates, low-quality articles, articles with no citations, and articles not written in English.

The main contributions of our proposal are the following:

- Taxonomize DevSecOps vulnerabilities identified by various experts. This helps researchers to be aware of what they need to secure, in every DevOps phase.
- Study the state-of-the-art DevSecOps and Zero Trust integration and analyze what is lacking.
- Introduce the idea of integrating Zero Trust into the stages of the legacy DevSecOps CI/CD pipeline using the NIST guidelines and analyze the literature on where this integration fits in.

Several recent papers review existing literature, focusing works on Zero Trust principles and ZTA(Zero Trust

Architecture) [Buck et al.(2021)], [Shin et al.(2025)]. Specifically, in [Buck et al.(2021)], the authors address the question of how to design and implement ZTA and summarize the findings about how Zero Trust adds value to organizations. In [Tak et al.(2017)], the authors present the fundamental tenets of Zero Trust and options for their realization. The paper discusses conventional approaches to authentication mechanisms, encryption, access control, micro-segmentation, software-defined perimeter, security automation, and the various associated challenges and presents the future direction for ZTA. Authors of [Fernandez and Brazhuk(2024)] present ZTA using security design patterns that can be used to form a reference architecture for ZTA implementations. Our paper, unlike the existing works in this area, provides a direction to the research community on looking at how the Zero Trust principles can be applied to the legacy DevSecOps stages and attempts to guide designers from an implementation perspective, by analyzing the shortcomings and merits of existing work and knowledge base.

The paper is structured as follows. Section 2 describes the essential background of DevOps, DevSecOps, and Zero Trust that is necessary to understand our survey paper. Section 3 proposes a stage-wise taxonomy for DevOps vulnerabilities, taking into account the existing research works in each of these broad areas. In Section 4, we discuss the integration of Zero Trust with DevSecOps using NIST guidelines. We consider the implementation of a device agent/gateway-based style of Zero Trust deployment for DevOps, which is particularly suited for distributed workforces, and discuss it from the point of view of the various requirements of Zero Trust. Section 5 provides a discussion of the challenges and promising research directions in this area of research. Finally, Section 6 concludes this work.

2. Background

This section provides the background knowledge necessary to understand the concepts used in the paper. Specifically, in the following, we briefly define the notions of DevOps, DevSecOps, and Zero Trust.

2.1. DevOps and DevSecOps

The term DevOps refers to a software engineering methodology designed in 2007, which integrates software development (Dev) and IT operations (Ops) to shorten the software development life cycle (SDLC) while providing continuous delivery, collaboration, correctness, and shared responsibility [Leite et al.(2019)]. Automation tools support the entire process, including building, configuration testing, continuous monitoring, and release. consists of the following steps: **Plan**, in this phase the teams define the objectives and the software requirements. **Develop**, application code is developed and properly versioned. **Build**, this phase deals with the building of executable files. **Test**, it consists of testing the application to ensure it meets functional, performance, and security requirements. **Release and Deliver**, consists of the code deployment into a repository accessible by the production servers for the deployment process. **Deploy**, the application is installed in the production environment. **Operate**, once

the application is deployed, it enters the operation phase, where the end users use, the software, and any issues that arise are addressed through maintenance and support activities. **Monitor**, occurs after the software has been deployed and is used regularly. It involves continuously monitoring infrastructure, processes, and applications to detect and respond to security threats and vulnerabilities in the production environment.

However, the intrinsic characteristics of such an approach, for example, frequent code changes and rapid deployment cycles, automation, and lack of proper security training, can increase the attack surface and introduce various security challenges [Lombardi and Fanton(2023)]. To tackle these issues, DevSecOps emerged as an extension of the DevOps practice incorporating security at scale. Since security and testing are moved to the left differently from the traditional linear depiction of SDLC, we refer to DevSecOps as the “shifting left” process.

This approach relies on a variety of technologies to allow automated security assessment [Fu et al.(2024)] as part of the CI/CD pipeline. Nonetheless, the integration of these technologies in a non-proprietary manner remains unresolved [Iannone and Palomba(2022)]. Some of the main procedures for obtaining security are the following: (i) analyzing software external components (i.e., third-party libraries) to identify associated vulnerabilities. (ii) Detecting insecure coding practices and known weaknesses in the source code. This procedure is called Static Application Security Testing (SAST). (iii) Monitor applications in run-time to find exploitable errors not detectable with static analysis and notify them. This process is known as Runtime Application Self-Protection (RASP). (iv) Using Web application Firewalls (or WAF) to monitor traffic. (v) Checking sensitive data (such as passwords or keys) accidentally committed and pushed to the code repository.

Two important concepts in CI/CD pipelines, which we will refer to in this paper, are the concepts of Containers and Microservices, both of which are employed in developing cloud applications. Containers are an encapsulation of software that contains all necessary components (the code and all its dependencies) for an application to run in any environment. It is an operating system-level virtualization. They are lightweight and can be rapidly deployed, patched, or scaled, and, therefore, they find use in agile and DevOps solutions to accelerate development, test, and production efforts.

Microservices are an architectural style in which an application is developed as a collection of services communicating over well-defined APIs. In large-scale software development projects, microservice architecture promotes modularity, agility, and scalability. Additionally, they are quickly deployable, improve fault isolation, improve team productivity, and increase cost efficiency. In the context of DevOps, microservices allow process streamlining to increase the productivity and quality of applications while moving to a more flexible architecture.

2.2. Zero Trust Architecture

Zero Trust Architecture (ZTA, in short) is an architectural framework where no user or device is implicitly trusted [Tak et al.(2017)]. Continuous authentication and access control require that every access request is verified

and authenticated regardless of whether it is related to a user, a device, or an application that relies on lightweight, scalable, and continuous authentication mechanisms. The principle of least privilege should be applied, granting users the minimum level of access required to perform their tasks. By creating microsegmentation, software can be isolated, and resources can be grouped (encapsulation). This helps minimize the amount of data that is exposed and reduces the risk of data breaches. Although encryption and data protection are important, the Zero Trust Architecture (ZTA) promotes its utilization to maintain the confidentiality and integrity of sensitive information. Furthermore, continuous monitoring ensures automated monitoring of compliance and risk issues.

The principles above enable Zero Trust Architecture (ZTA) to improve threat detection and response and reduce attack surface area, thereby safeguarding the security and privacy of data. The main components of ZTA, sketched in Figure 1, are (i) a **Policy Engine (PE)** in charge of making the access decision following enterprise policies; (ii) a **Policy Administrator (PA)**, which permits or denies access creating session tokens; (iii) a **Policy Enforcement Point (PEP)**, which manages and monitors the connection between the subject and the resource; (iv) several **data sources**, internal and external sources of data that provide input and policy rules used by PE for making access decisions. The Policy Engine and the Policy Administrator together form the **Policy Decision Point (PDP)**. In figure PDP is the Policy Definition Point, which makes decisions about access requests within the network. The PDP is a **Control Plane** component, which sets rules and policies that govern access; while the PEP is a **Data Plane** component, that executes the policies established by the Control Plane and facilitates secure data exchange.

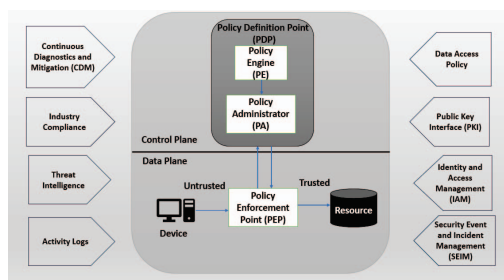


Figure 1. Zero Trust Architecture main components.

3. Vulnerabilities in DevOps

In this section, we create a taxonomy of the vulnerabilities that exist in **DevOps systems** and group them into eight broad categories based on “where” they are exposed (as in Figure 2). These vulnerabilities are partially addressed in the DevSecOps pipeline but have scope for further improvement. We believe that such a taxonomy would help researchers apply security measures appropriately to the different phases of **DevSecOps**. In this context, we also examine some existing mitigation methods and research papers in connection with the category.

3.1. Code and test data security

Developer code and test data are to a large extent, confidential assets that belong to organizations. Attackers may compromise a developer’s login credentials to get access to the development machine and steal the source code and data. Disgruntled, unethical, and careless employees may also leak this information to the public for personal gain. Many developers leave security keys and authentication details hardcoded in the source code, providing attackers with backdoor access to the systems.

Examples of various issues in access control of code management infrastructure are: (i) a fraudulent user can write malicious code to the code repository through loose permissions; (ii) there could be a breach of low-security service that has permissions to modify source code; and (iii) the developer credentials can be stolen [Vehent(2018)]. These issues arise primarily in the “Develop” stage of DevSecOps. Organizations can ensure that their assets are not tampered with, by enforcing secure coding practices, access control, encryption, clear incidence response plans, off-site backups, secure code repositories, secure end-point devices, monitoring, and audits.

3.2. Build cycle security

The “Build” stage in the DevSecOps cycle gives third-party automated build tools privileged access to the code base. Dependencies, such as third-party libraries or frameworks, used in the build may also introduce vulnerabilities. It’s essential to review and scan these dependencies for security issues during the build phase. Tools like OWASP Dependency-Check¹, and SonarQube² analyze dependencies for vulnerabilities. To deal with Build cycle vulnerabilities, Maturity Models like OWASP’s SAMM model³ emphasize building software in a standardized, repeatable manner using secure components that include open-source tools and dependencies.

3.3. Vulnerabilities in DevOps tools

Automation and tools are a major consideration in DevOps [Lombardi and Fanton(2023)]. Automation tools are mostly open-source and enable core DevOps practices, such as testing automation, versioning, continuous integration, and configuration management. DevOps tools, in different stages of the DevOps cycle, have complete access to the repositories.

Authors of [Kumar and Goyal(2020)] define a workflow, that introduces security controls in DevOps processes. Improved tooling is the first step to successful DevOps. Management of users in third-party tools such as GitHub, can help reduce vulnerabilities due to the DevOps tools. Precautionary steps like keeping the list of GitHub owners as small as possible, requiring Multi-Factor Authentication (MFA), and regularly auditing members of the GitHub organization are required. Recommended practices, such as using the OAuth2 authorization framework and taking precautions to ensure users who delegate

1. <https://owasp.org/www-project-dependency-check/>
2. <https://www.sonarsource.com/products/sonarcloud/>
3. <https://owaspsamm.org/model/>

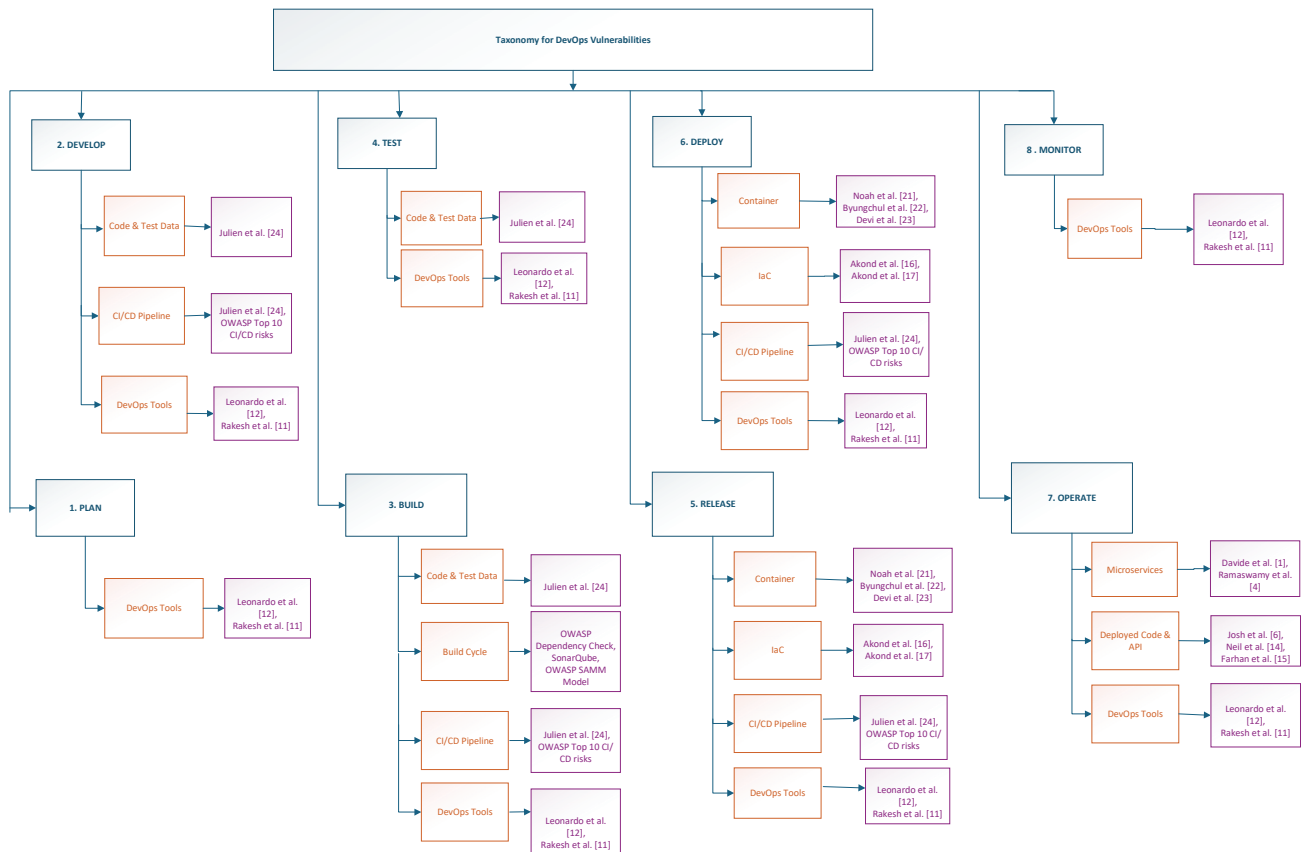


Figure 2. Taxonomy of DevOps Vulnerabilities.

permissions aren't aware of the repository organization, whitelisting authorized third parties, and considering splitting GitHub organizations to compartmentalize sensitive applications, are enforced.

3.4. Container security breaches

Continuous delivery/deployment of applications, specifically microservices, uses the capability and automation of containers and container management frameworks like Kubernetes. Attackers may access container repositories that store images to violate the image during the "Release" and "Deploy" stages of DevSecOps.

There are several reasons for vulnerabilities to appear in containers [Tak et al.(2017)], namely (i) it is difficult to establish a full pedigree of images downloaded from public registries that may have vulnerabilities introduced unintentionally via updates by various users; (ii) in the occasion of non-conformance to immutable software deployment policies (like those promoted by DevOps principles), vulnerabilities are introduced, giving way to loss of control over deployed software. Hardening, compliance verification, and runtime monitoring in cloud-native applications are methods that fully support the complete life cycle of containerized applications. As a solution, there are several tools for assessing container vulnerabilities [VS et al.(2023)]. For example, the well-known threat modeling framework, DREAD (Damage, Reproducibility, Exploitability, Affected Users, and Discoverability), can be utilized for advanced threat modeling to contrast, eval-

uate, and order the magnitude of threats associated with containers. Research has been conducted on Kubernetes, Docker, and workflow tools and a high-interaction honeypot has been designed in [Spahn et al.(2023)] to reveal where attackers are prone to strike and what is being done against exposed instances.

3.5. IaC security breaches

Infrastructure as Code (IaC) is a key feature in the DevOps "Release" and "Deploy" stages, that allows practitioners to deploy servers to different environments. When developing these scripts, practitioners may inadvertently introduce security "smells", which are recurring coding patterns that lead to security breaches. IaC scripts introduce various smells such as: (i) specifying default users as admins; (ii) using a string of zero length as a password; (iii) hard-coding usernames and passwords; (iv) binding invalid IP addresses to servers; (v) putting revealing information in comments; (vi) using HTTP without TLS; and (vii) using weak cryptographic algorithms [Rahman et al.(2019)]. Several solutions are discussed in the literature to deal with IaC security breaches. Practitioners may inspect the presence of these security smells in Ansible and Chef scripts using code review and static analysis tools [Rahman et al.(2021)].

3.6. Vulnerabilities in CI/CD pipeline

CI/CD is one of the most popular practices software developers use to deliver code reliably. The security of

outsourced DevSecOps pipelines can be quite weak [Vehent(2018)] and OWASP's top 10 CI/CD security risks⁴, which broadly covers all of the vulnerabilities discussed in this section, has special relevance to CI/CD pipeline vulnerabilities. The lack of security of default configurations in pipelines can be due to elevated permissions for pipeline components. Instead of network isolation, it is recommended that we move to a model where access control secures the pipeline.

3.7. Vulnerabilities in micro-services

Here, the application is broken down into loosely coupled components that interact with each other over a network. The most commonly tackled attacks in applications during the "Operate" stage of DevSecOps, using micro-services, are spoofing and Denial Of Service as a result of the fine granularity, flexible scalability, and independence of services [Berardi et al.(2022)]. Some of the existing mitigation approaches for dealing with vulnerabilities in micro-services are to implement access control, user authentication, encryption, and API protection [Chandramouli(2022)].

3.8. Vulnerabilities in deployed code and APIs

Cloud-native applications in production reside in the Cloud Service Provider (CSP, for short) infrastructures. The CSPs are responsible for securing the infrastructure and storage in the platform where the application is built. However, depending on how the application functions, how the access control is configured, and what it exposes online, there is plenty of room for the application in the "Operate" stage of DevSecOps, to expose security risks.

Data at rest is usually not encrypted by default in cloud platforms. Cloud misconfigurations could also lead to exposing data storage and thereby creating vulnerable workloads. Other issues like broken access control issues, leaked secrets, unpatched vulnerabilities, and leaky APIs open up further scope for attacks in production.

Production web servers are often exposed to various types of attacks such as Denial of Service(DoS), Distributed Denial of Service(DDoS), SSH brute force attack, Cross-Site Scripting(XSS), SQL Injection attack, Man-in-the-middle(MITM) attack and so on, to name a few. Mitigation strategies involve strategies like regular updates and patching, Web Application Firewalls(WAF), Multi-Factor Authentication(MFA), access control, encryption, secure coding practices, input validation and sanitization, security training, incidence response plans, monitoring and logging, data backups, and physical security.

APIs are the most vulnerable attack points, and most users are unaware of API insecurity and lack adequate API security training [Qazi(2022)]. Equifax's high-profile data breach, which exploited a vulnerability in the open source API component⁵, was one of the worst data breaches in recent times. The most serious attacks in API are through SQL injection attacks [Madden(2020)], which

can be prevented by validation of inputs. APIs expose end-points of object-level identifiers and create a wide attack surface of object-level access control issues. As with micro-services, the existing mitigation approaches for dealing with vulnerabilities in APIs are to implement tight user authentication, encryption, and API protection.

4. Zero Trust and DevSecOps integration

Massive digital transformations like cloud adoption, container technology, and distributed infrastructure, made existing security controls less effective. To address the vulnerabilities discussed in the previous section, a framework that shifts the paradigm towards Zero Trust is needed, focusing on tighter authentication and authorization, encryption, monitoring, and network segmentation. In recent years, there has been an increase in industry and academia, in combining the areas of Zero Trust and DevSecOps [Tak et al.(2017)], [Chandramouli(2022)]. Zero Trust is a proactive method that continuously verifies access requests, making it a framework that aligns perfectly with the DevSecOps culture. In this section, we contextualize and prioritize existing research in the integration of DevSecOps with Zero Trust. We also suggest implementing a ZTA model based on the NIST guidelines to achieve Zero Trust in a DevSecOps environment.

4.1. State-of-the-art Survey

To combine Zero Trust as a security strategy with DevSecOps as a development strategy, the authors of [Sanders et al.(2021)] use a mission thread to provide context to the discussion of an application's life cycle as it goes through a DevSecOps pipeline. The Zero Trust considerations adopted by the mission thread through various phases show the enforcement of Zero Trust tenets. This research lays out a foundation for making the Zero Trust transition possible but does not contribute to the architecture of the integration.

The industry has turned to using service mesh for cloud-native application development using Zero Trust principles. The core tenets of security, reliability, and observability enable scalability, testability, and flexibility in the DevSecOps stages of designing, developing, building, testing, deploying, operating, and monitoring software in the cloud [Chandramouli(2022)]. One such use case of the Istio service mesh⁶ for DevSecOps utilizes the open-source distributions of Envoy (proxy), Spiffe/Spire (identity framework), and Open Policy Agent (general-purpose policy engine) to implement Zero Trust via an mTLS communication channel [Hightower(2022)]. This architecture is designed specifically to work for Kubernetes deployments using microservices and does not suit legacy applications.

In [Shin et al.(2025)], authors build a framework based on Zero Trust Architecture for microservices-based DevOps. The framework also combines service mesh technologies into modern DevOps implementations. However, implementations based on this framework help cloud nativization of DevSecOps and not legacy DevSecOps.

4. <https://owasp.org/www-project-top-10-ci-cd-security-risks/>

5. www.csoonline.com/article/567833/equifax-data-breach-faq-what-happened-who-was-affected-what-was-the-impact.html

6. <https://istio.io>

4.2. Migration to ZTA using NIST Guidelines

NIST provides a comprehensive architecture for implementing ZTA in different deployment scenarios [Stafford(2020)], which could be leveraged for a DevSecOps pipeline. The different deployment models of the abstract ZTA architecture, viz. Device Agent/Gateway model, Enclave Gateway Model, Resource Portal Model, and Application Sandbox model [Garbis and Chapman(2021)] offer guidelines for deployment. We hereby call out that for a DevSecOps CI/CD pipeline, as illustrated in Figure 3, an implementation model such as the Device Agent/Gateway, as shown in Figure 4, serves the purpose of delineating the repositories from open access. The advantage of applying the Device Agent/Gateway model to the DevSecOps pipeline is that the interactions between the elements of the pipeline are delineated by continuous access control at each layer, which allows or denies specific users from running certain operations and protects the code that transits from the developers to the production environment. Subsequently, we describe how this model is fully compliant with Zero Trust principles.

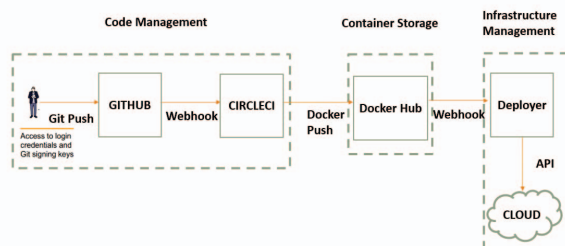


Figure 3. CI/CD Pipeline from [Vehent(2018)].

The following Zero Trust requirements are applied to the integration framework in Figure 4:

- **Micro-segmentation:** Micro-segmentation isolates networks and the trust zones so that the attack surface is marginally reduced in a network [Tak et al.(2017)]. By separating the components of DevSecOps into different workloads and defining the trust boundaries via placement of security gateways or Next Generation Firewalls that act as Policy Enforcement Points (PEP), near the resources, the Zero Trust architecture makes it difficult for attackers to laterally move to high-security resources.
- **Continuous and Dynamic Authentication and Access Control:** Authentication and authorization to resources in the DevSecOps systems would be via certificate exchange and dynamic security policies enforced in the Policy Definition Point (PDP), which makes sure that the users/applications and devices (the subjects) are authenticated for every resource request and rules for access to resources are governed by the dynamic policies. MFA is employed for authentication, which adds one more layer of security by requiring users to provide multiple forms of verification.
- **Encryption:** One of the goals of Zero Trust is to protect the data, either at rest or in transit.

Advanced encryption techniques are used in the DevSecOps pipeline to further enhance data protection and privacy in the DevSecOps pipeline.

- **Automation:** An ideal Zero Trust implementation automates every function of the control plane and the data plane of the ZTA. The brain of the ZTA is the AI-driven Trust Algorithm that resides in the PDP. Apart from trust evaluation and access control, attack detection and resource monitoring are also driven by automated AI algorithms [Cao et al.(2024)]. The AI algorithms use sources like Identity and Access Management (IAM), Multi-factor Authentication (MFA), Mobile Device Management (MDM), Security Information and Event Management (SIEM), and Governance, Risk and Compliance (GRC) to make access decisions (ref. Figure 4).
- **Continuous Monitoring:** The continuous monitoring of user behavior, device health, and network traffic through event log collection allows anomaly detection that helps respond quickly to security events.

In the following, we discuss how Zero Trust can be applied to each stage of the DevSecOps life cycle and analyze areas in different research works where Zero Trust would fit in. By combining these strategies, organizations can significantly enhance their security posture to reduce the risk of security breaches.

4.2.1. Plan. Zero Trust's least-privilege access control can be applied to the tools and systems used in the project planning by implementing the principle of least privilege, micro-segmentation, and dynamic policies for authentication, and access control.

4.2.2. Develop. Restricted access to code repositories needs to be implemented by fine-grained access control to overcome the vulnerabilities discussed in Sections 3.1, 3.3. Apart from implementing secure coding practices, enforcing the least privileged access control helps ingrain security into the development environment. Integrating Zero Trust into the development phase involves applying micro-segmentation and dynamic policies for authentication and access control to restrict access to code repositories. Micro-segmentation imposed by the PEP agent and gateway in the development environment in Figure 4 forces the developers to access the code and test data in a Zero-Trust manner.

4.2.3. Build. The vulnerabilities discussed in Sections 3.2, 3.6, 3.3, would be addressed only if there is a separation of function between the Github development environment and the CircleCI or Maven Build environment, which could be accomplished by applying micro-segmentation (see Figure 4). In this phase, where the application is compiled and packaged for deployment, Zero Trust principles can be applied by ensuring that only authorized personnel/tools have access to the built environment and that the build process is secure. Code repositories are accessed by open-source automated build tools. Policy enforcement for fine-grained access control should be enforced so that automated requests come only

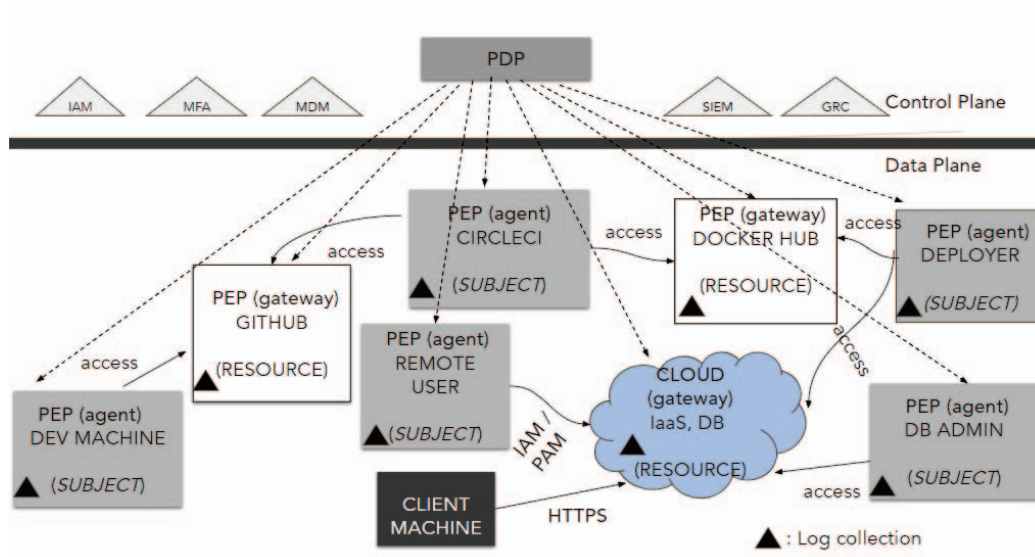


Figure 4. CI/CD pipeline using NIST's Device Agent/Gateway Model for Zero Trust.

from trusted devices. Passwordless and keyless authentication in Zero Trust ensures that humans who configured the automated processes don't retain access to their credentials. This reduces the risk of credential theft, unauthorized access, and potential security breaches within the Zero Trust environment.

4.2.4. Test. Automated security tools that access the DevOps systems should not have access to secure data as discussed in Section 3.1. Zero Trust policy enforcement should be enforced in this layer too. Zero Trust also lends a completely automated environment for continuous testing.

4.2.5. Release and Deliver. Vulnerabilities as discussed in Sections 3.4, 3.5, 3.6 may be exposed in this stage if proper security measures are not enforced. Zero Trust can be applied in this stage by separating (as shown in Figure 4) the CircleCI/Maven build environments from the DockerHub environments where the Containers are released, and ensuring that the least privileged access is applied to access images and environment configuration files, and that security testing is seamlessly automated, as in the testing phase.

4.2.6. Deploy. Zero Trust ensures that only authorized personnel and/or tools can deploy the application and that the deployment process is secure. Micro-segmentation, dynamic and continuous authentication and authorization, and continuous monitoring and event logging should be applied. Infrastructure in the cloud, where the application is deployed, should also go through security checks. In Figure 4, the Deployer is separated from the Cloud environment via Zero Trust micro-segmentation to ensure that the IaC and Container environments do not get exposed to vulnerabilities as discussed in Sections 3.4, 3.5, 3.6.

4.2.7. Operate. The vulnerabilities discussed in Sections 3.8, 3.7 have to be addressed and Zero Trust principles can be applied in the "Operate" stage through micro-segmentation to the systems accessing the Cloud environ-

ment (as shown in figure 4), and by continuously monitoring the devices, applications, and user access, for any security threats and responding quickly to any identified threats. Remote users access resources through Identity and Access Management (IAM) and/or Privileged Access Management (PAM) access control. Data encryption is another feature of Zero Trust systems that needs to be applied to prevent DoS, malware, database invasion, unauthorized internet access, etc.

4.2.8. Monitor. Zero Trust can be applied by implementing continuous security monitoring and responding quickly to security alerts. Integrating cybersecurity monitoring into DevSecOps monitoring requires developers and operators to be involved in configuring their monitoring and alerting services. Zero Trust security monitoring could be incorporated to bring automation to the process of inspection of network traffic, user behavior, and device activities.

5. Challenges and Research Directions

DevSecOps and Zero Trust frameworks are now starting to be adopted in organizations. In the following, we list the major existing challenges that provide open research directions to future investigations.

- **Model deployment.** Implementing a ZTA deployment model in the DevSecOps pipeline may involve a high degree of automation, and such an implementation cannot be used in all types of systems. Furthermore, there is a possibility of pushback from the application owners when it comes to agent deployments in application servers. Agentless models may be a better choice in such situations [Garbis and Chapman(2021)]. Remote access capability for BYOD (Bring Your Own Device) into a resource in the Zero Trust enabled DevSecOps pipeline is also a challenge. The areas of ZTA deployment models for DevSecOps and its

landscape such as its application to IoT application development and API development should be further investigated.

- **Scalability.** Scalability and adaptability for large-scale deployments and defining metrics to evaluate the performance of a Zero Trust implementation in DevSecOps, is another important area for future research.
- **Continuous monitoring.** Traditional logging methods collect messages from various sources on a central server. However, modern logging methods [Vehent(2018)] involve a complex logging pipeline that leaves scope for future research. Such monitoring techniques are more complex for IoT applications because of the need for a central server that collects logs from many endpoints for analysis. In the DevSecOps scenario, the type of information that needs to be logged is also relevant. For example, trust is violated if an automated process kicks off another automated process requesting data.
- **User experience.** For DevSecOps systems that need always-on access for developers, Zero Trust tends to make it difficult by enforcing continuous authentication. The user experience in the various Zero Trust deployment models for DevSecOps needs to be evaluated to understand the implications of the practicality of the solutions.

6. Conclusion

This paper provides insight into the different security challenges of DevSecOps and studies the importance of an extreme shift-left approach such as Zero Trust. This security concept can be beneficial for DevSecOps because it assumes threats could be both external and internal, and thus, no entity should be automatically trusted. Qualitative analysis of research publications has shown that there is a gap in the literature in studying implementations of Zero Trust Architectures to cover the entire DevSecOps pipeline. In this paper, we discuss the integration of NIST's Device Agent/Gateway deployment model into the DevSecOps pipeline and consider it from the point of view of each DevSecOps stage. We discussed the various open research areas that can uncover the scope for practical implementations of the proposed integration. We argued that by integrating DevSecOps into the Zero Trust model, organizations can strengthen their security posture and better protect against advanced threats, particularly in today's environment where traditional perimeter-based defenses are no longer sufficient due to factors like cloud computing, Internet of Things, Web Services, and mobile devices.

References

- [Berardi et al.(2022)] Davide Berardi, Saverio Giallorenzo, Jacopo Mauro, Andrea Melis, Fabrizio Montesi, and Marco Prandini. 2022. Microservice security: a systematic literature review. *PeerJ Computer Science* 8 (2022), e779.
- [Buck et al.(2021)] Christoph Buck, Christian Olenberger, André Schweizer, Fabiane Völter, and Torsten Eymann. 2021. Never trust, always verify: A multivocal literature review on current knowledge and research gaps of zero-trust. *Computers & Security* 110 (2021), 102436.
- [Cao et al.(2024)] Yang Cao, Shiva Raj Pokhrel, Ye Zhu, Robin Doss, and Gang Li. 2024. Automation and orchestration of zero trust architecture: Potential solutions and challenges. *Machine Intelligence Research* 21, 2 (2024), 294–317.
- [Chandramouli(2022)] Ramaswamy Chandramouli. 2022. Implementation of devsecops for a microservices-based application with service mesh. *NIST Special Publication* 800 (2022), 204C.
- [Fernandez and Brazhuk(2024)] Eduardo B Fernandez and Andrei Brazhuk. 2024. A critical analysis of Zero Trust Architecture (ZTA). *Computer Standards & Interfaces* 89 (2024), 103832.
- [Fu et al.(2024)] Michael Fu, Jirat Pasuksmit, and Chakkrit Tantithamthavorn. 2024. Ai for devsecops: A landscape and future opportunities. *ACM Transactions on Software Engineering and Methodology* 1 (2024), 1557–7392.
- [Garbis and Chapman(2021)] Jason Garbis and Jerry W Chapman. 2021. *Zero Trust Security: An Enterprise Guide*. Springer, Apress Berkeley, CA.
- [Hightower(2022)] Kelsey Hightower. 2022. Zero Trust the Hard Way. <https://www.youtube.com/watch?v=PMhPWGRzIzM>
- [Iannone and Palomba(2022)] Emanuele Iannone and Fabio Palomba. 2022. The Phantom Menace: Unmasking Security Issues in Evolving Software. In *2022 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, Limassol, Cyprus, 612–616.
- [Kumar and Goyal(2020)] Rakesh Kumar and Rinkaj Goyal. 2020. Modeling continuous security: A conceptual model for automated DevSecOps using open-source software over cloud (ADOC). *Computers & Security* 97 (2020), 101967.
- [Leite et al.(2019)] Leonardo Leite, Carla Rocha, Fabio Kon, Dejan Milojevic, and Paulo Meirelles. 2019. A survey of DevOps concepts and challenges. *ACM computing surveys (CSUR)* 52, 6 (2019), 1–35.
- [Lombardi and Fanton(2023)] Federico Lombardi and Alberto Fanton. 2023. From DevOps to DevSecOps is not enough. CyberDevOps: an extreme shifting-left architecture to bring cybersecurity within software security lifecycle pipeline. *Software Quality Journal* 31, 2 (2023), 619–654.
- [Madden(2020)] Neil Madden. 2020. *API security in action*. Simon and Schuster, New York, USA.
- [Qazi(2022)] Farhan A Qazi. 2022. *Insecure application programming interfaces (apis) in zero-trust networks*. Capitol Technology University, Washington D.C., USA.
- [Rahman et al.(2019)] Akond Rahman, Chris Parnin, and Laurie Williams. 2019. The seven sins: Security smells in infrastructure as code scripts. In *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*. IEEE, Montréal, QC, Canada, 164–175.
- [Rahman et al.(2021)] Akond Rahman, Md Rayhanur Rahman, Chris Parnin, and Laurie Williams. 2021. Security smells in ansible and chef scripts: A replication study. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 30, 1 (2021), 1–31.
- [Rajapakse et al.(2022)] Roshan N Rajapakse, Mansoor Zahedi, M Ali Babar, and Haifeng Shen. 2022. Challenges and solutions when adopting DevSecOps: A systematic review. *Information and software technology* 141 (2022), 106700.
- [Sanders et al.(2021)] Geoffrey Sanders, Timothy Morrow, Nataniel Richmond, and Carol Woody. 2021. Integrating zero trust and DevSecOps.
- [Shin et al.(2025)] Daemin Shin, Jiyeon Kim, I Wayan Adi Juliawan Pawana, and Ilun You. 2025. Enhancing cloud-native DevSecOps: A Zero Trust approach for the financial sector. *Computer Standards & Interfaces* 93(103975) (2025), 103975.
- [Spahn et al.(2023)] Noah Spahn, Nils Hanke, Thorsten Holz, Christopher Kruegel, and Giovanni Vigna. 2023. Container orchestration honeypot: Observing attacks in the wild. In *Proceedings of the 26th International Symposium on Research in Attacks, Intrusions and Defenses*. ACM New York, NY, Hong Kong, China, 381–396.
- [Stafford(2020)] V Stafford. 2020. Zero trust architecture. *NIST special publication* 800, 207 (2020), 800–207.

- [Tak et al.(2017)] Byungchul Tak, Canturk Isci, Sastry Duri, Nilton Bila, Shripad Nadgowda, and James Doran. 2017. Understanding security implications of using containers in the cloud. In *2017 USENIX Annual Technical Conference (USENIX ATC 17)*. Usenix association, Santa Calsa, CA, USA, 313–319.
- [Vehent(2018)] Julien Vehent. 2018. *Securing DevOps: security in the cloud*. Simon and Schuster, New York, USA.
- [VS et al.(2023)] Devi Priya VS, Sibi Chakkaravarthy Sethuraman, and Muhammad Khurram Khan. 2023. Container security: precaution levels, mitigation strategies, and research perspectives. *Computers & Security* 135 (2023), 103490.